

Ideation Phase

Brainstorm & Idea Prioritization

Date	30 September 2026
Team ID	SWTID-2026-8273
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Project Context

The ServiceNow Incident form is used by the Service Desk to log and resolve user issues. Without form-level controls, agents can save incomplete records (for example, a Resolved incident with no close notes), see fields that are irrelevant to the current State or Category, and re-enter data the platform already knows. This project uses Client Scripts and UI Policies to guide agents at the point of entry and improve data quality.

Team Gathering & Collaboration

- The team met to align on the project goal, scope (Incident table only) and the two tools in focus: UI Policies (no-code, field behaviour) and Client Scripts (JavaScript, onLoad / onChange / onSubmit).
- Ground rules: quantity over quality, no judging ideas during listing, build on each other's ideas, and stay within what a ServiceNow Administrator can configure without changing the data model.
- Each member listed ideas independently, then the team grouped similar ideas and scored them together.

Selected Problem Statement

Problem Statement
How might we make the Incident form guide agents to enter complete, accurate and consistent data (mandatory fields, auto-filled values, and only the fields relevant to the current Category and State) using Client Scripts and UI Policies, without slowing down ticket handling?

Step-2: Brainstorm, Idea Listing and Grouping

Ideas were generated individually and then grouped by the type of form behaviour they control. Each idea lists the ServiceNow tool best suited to deliver it.

#	Idea / Use Case	Description & Example Behaviour	ServiceNow Tool
1	Mandatory Field Validation	Make key fields mandatory at the right moment. • Short description always mandatory. • State = Resolved / Closed: Close code and Close notes mandatory. • State = On Hold: On hold reason mandatory. • Block save with an error message if a required field is empty.	UI Policy (Make mandatory) + Client Script (onSubmit)
2	Dynamic Show / Hide of Fields	Show only what is relevant. • Show On hold reason only when State = On Hold. • Show Resolution details (Close code, Close notes) only when State = Resolved / Closed. • Show Configuration item / Work notes prompts based on Category (e.g., Hardware, Network).	UI Policy (Visible / Hidden)
3	Auto-Populate Fields	Reduce manual entry and errors. • On Caller change, auto-fill Location and Contact details from the user record. • Default Contact type and set Assignment group hint from Category / Subcategory. • Clear Subcategory when Category changes.	Client Script (onChange / onLoad, GlideAjax or getReference)
4	Read-Only Lock on Closed Incidents	Protect finished records. • State = Closed: make Caller, Category, Impact, Urgency and Short description read-only. • Keep Priority read-only (calculated from Impact and Urgency) to avoid manual overrides.	UI Policy (Read-only) + onLoad Client Script
5	Real-Time User Guidance & Alerts	Help agents make better decisions. • Info message when Impact = 1 and Urgency = 1 (Priority 1 – Critical): "Notify the Major Incident Manager". • onSubmit confirmation before saving a P1 incident. • Field hint under Short description for minimum detail.	Client Script (onChange, onSubmit)

Step-3: Idea Prioritization

Each idea was scored on two criteria using a 1–5 scale. Impact is the improvement in data quality and agent productivity. Feasibility is how easy the idea is to build and test as an Administrator (5 = very easy, 1 = very hard). Priority Score = Impact × Feasibility.

Impact vs Effort / Feasibility Scoring

#	Idea	Impact (1-5)	Feasibility (1-5)	Score	Effort	Rank
1	Mandatory Field Validation	5	5	25	Low	1
2	Dynamic Show / Hide of Fields	4	5	20	Low	2
3	Auto-Populate Fields	4	4	16	Medium	3
4	Read-Only Lock on Closed Incidents	3	5	15	Low	4
5	Real-Time User Guidance & Alerts	3	4	12	Medium	5

Impact vs Effort Matrix

	Low Effort (High Feasibility)	Medium Effort
High Impact	QUICK WINS – Do first Idea 1: Mandatory Field Validation Idea 2: Dynamic Show / Hide	PLAN & BUILD – Do next Idea 3: Auto-Populate Fields
Moderate Impact	FILL-INS – If time permits Idea 4: Read-Only Lock on Closed	BACKLOG – Later Idea 5: Guidance & Alerts

Ideas Selected for Implementation

Rank	Selected Idea	Reason for Selection	Build Phase
1	Mandatory Field Validation	Highest score; directly prevents incomplete incidents and is simple to build with a UI Policy plus an onSubmit Client Script.	Phase 1
2	Dynamic Show / Hide of Fields	High value with no code; keeps the form clean and shows fields only for the relevant State / Category.	Phase 1
3	Auto-Populate Fields	Saves agent time and reduces data-entry errors; needs an onChange Client Script and testing of the reference lookup.	Phase 2

Not selected now: Idea 4 (Read-Only Lock) can be added as a small extension of Idea 2 if time permits. Idea 5 (Guidance & Alerts) is kept in the backlog as it adds convenience but has the lowest impact on data quality.